Reviewer Pack — Minimal Rank of Quantum Group Cohomology vs Resolution Proof…

Entry 31d289fa7b42 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 22:26:43 UTC

Minimal Rank of Quantum Group Cohomology vs Resolution Proof Length

Entry ID: `31d289fa7b42` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 22:26:43 UTC

1. Conjecture

Field A (mathematical branch): Quantum Groups **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For any Tseitin formula F on n variables, the minimal rank of the quantum group cohomology of the associated graph G is lower bounded by $2^{\Omega(\log}2(n/\delta))$, where δ is the minimum number of clauses per variable in F.', 'This lower bound holds for all instances of Tseitin formulas with $n \leq 40$ and $\delta \leq 5$.']

Rationale (proposer's reasoning):

> ['Quantum group cohomology has been used to study properties of graphs, but its application to complexity theory is rare. This conjecture suggests that the structure encoded in quantum group cohomology could be related to the difficulty of solving Tseitin formulas, potentially revealing new insights into resolution proof complexity.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `87a3731d6007f32a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the minimal rank of quantum group cohomology for all Tseitin formulas meets or exceeds the lower bound of $2^{\Omega(\log}2(n/\delta))$ with at least 80% of the seeds, and falsified if any seed results in a minimal rank below this bound.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n, delta):
        variables = list(range(1, n + 1))
        clauses = []

        for var in variables:
            clause = [random.choice([-1, 1]) * var]
            for _ in range(delta - 1):
                other_var = random.choice(variables)
                if other_var != var:
                    clause.append(random.choice([-1, 1]) * other_var)
            clauses.append(clause)

        return variables, clauses

    def tseitin_formula_to_graph(variables, clauses):
        graph = {var: set() for var in variables}

        for i, clause in enumerate(clauses):
            for j in range(len(clause)):
                for k in range(j + 1, len(clause)):
                    var1 = abs(clause[j])
                    var2 = abs(clause[k])
                    graph[var1].add(-var2)
                    graph[-var1].add(var2)

        return graph

    def quantum_group_cohomology_rank(graph):
        n = len(graph)
        adjacency_matrix = [[0] * n for _ in range(n)]
```

```
    for var, neighbors in graph.items():
        index = abs(var) - 1
        for neighbor in neighbors:
            if neighbor > 0:
                adjacency_matrix[index][abs(neighbor) - 1] += 1

    # Gaussian elimination to find the rank of the matrix
    row echelon_form = [row[:] for row in adjacency_matrix]
    rank = n

    for i in range(n):
        pivot_row = next((j for j in range(i, n) if echelon_form[j][i] != 0), None)
        if pivot_row is None:
            rank -= 1
            continue

        # Swap rows to put the pivot at the current position
        echelon_form[i], echelon_form[pivot_row] = echelon_form[pivot_row], echelon_form[i]

        # Make all entries below the pivot zero
        for j in range(i + 1, n):
            factor = echelon_form[j][i] / echelon_form[i][i]
            for k in range(n):
                echelon_form[j][k] -= factor * echelon_form[i][k]

    return rank

def resolution_proof_length(clauses):
    # Simplified DPLL solver to estimate proof length
    stack = []
    assignment = {}

    def dpll():
        if not clauses:
            return 0

        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            assignment[abs(literal)] = literal > 0
            clauses = [c for c in clauses if literal not in c and -literal not in c]
            return dpll()

        var = next((v for v in range(1, len(variables) + 1) if v not in assignment), None)
        stack.append((-var, assignment.copy()))
        assignment[var] = True
        clauses = [c for c in clauses if var not in c and -var not in c]
        proof_length = dpll()
        if proof_length is not None:
            return proof_length + 1

        stack.pop()
        assignment[var] = False
        clauses.append([-var])
        proof_length = dpll()
        if proof_length is not None:
```

```
                return proof_length + 1

            stack.pop()
            return None

        return dpll()

    n = random.randint(5, 40)
    delta = random.randint(2, 5)
    variables, clauses = generate_tseitin_formula(n, delta)
    graph = tseitin_formula_to_graph(variables, clauses)
    rank = quantum_group_cohomology_rank(graph)
    proof_length = resolution_proof_length(clauses)

    metric_name = "minimal_rank"
    metric_value = rank
    instances_tested = 1
    conjecture_holds = rank >= 2 ** (math.log(n / delta, 2) ** 2)
    counterexample = "" if conjecture_holds else f"n={n}, delta={delta}, rank={rank}"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) + [101, 10

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.2:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
le': ''}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 43705.84586200712, 'instances_tested': 30, 'c
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11817039.206880739, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 23594761.727919333, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 18522.678880868432, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 13006978.614017108, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11816531.713677224, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 650791.6487459983, 'instances_tested': 30, 'c
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 47200.20333599558, 'instances_tested': 30, 'c
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11827449.130012719, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 12995139.092288826, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 12978676.587484406, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11216058.898070786, 'instances_tested': 30, '
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 594683.0922487624, 'instances_tested': 30, 'c
RESULT: SUPPORTED mean=10777727.85241576 std=8537405.742461903 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The conjecture is supported by testing only n ≤ 40, which is too small to establish a general trend. The metric does not scale trivially with n, but the small size of tested instances (n ≤ 40) may not be sufficient to confirm the conjecture's validity for larger values of n.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge | original judge: The test results show that the minimal rank of quantum group cohomology meets or exceeds the lower bound for all tested instances with a support fract | next: Further testing with larger values of n (greater than 40) and δ to confirm the conjecture's validity for a wider range of Tseitin formulas.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11052
2	preregistration	ollama_remote	glm4:latest	0	0	5594
3	novelty	ollama_remote	glm4:latest	0	0	4604
4	novelty	ollama_remote	glm4:latest	0	0	5194
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41350
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15343
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8944
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15279
9	critic	ollama_remote	glm4:latest	0	0	9074
10	judge	ollama_remote	glm4:latest	0	0	5747

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 122181 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/31d289fa7b42.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/31d289fa7b42.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/31d289fa7b42.tar.gz` (if generated)